
NISAAP Platform

Technical Documentation

Network Infrastructure Security Assessment & Action Platform

Organization: National Railways of Zimbabwe

Division: IT Security Division

Version: 1.0.0

Date: June 2026

Enhancing Network Security in IoT Devices at NRZ

Table of Contents

- 1. Introduction..... 1
 - 1.1 Purpose of This Document.....1
 - 1.2 Platform Overview.....2
 - 1.3 Key Features Summary..... 2
- 2. System Architecture..... 2
 - 2.1 High-Level Architecture..... 2
 - 2.2 Technology Stack.....3
 - 2.3 Deployment Architecture.....3
- 3. Database Design.....3
 - 3.1 Database Technology..... 3
 - 3.2 Data Model.....4
 - 3.2.1 User Model..... 4
 - 3.2.2 IoTDevice Model.....4
 - 3.2.3 Vulnerability Model.....4
 - 3.2.4 SecuritySolution Model.....5
 - 3.2.5 Assessment Model.....5
 - 3.2.6 AuditLog Model.....5
 - 3.3 Entity Relationships.....5
- 4. API Endpoints..... 6
 - 4.1 API Architecture.....6
 - 4.2 Authentication Endpoints.....6
 - 4.3 Device Endpoints..... 6
 - 4.4 Vulnerability Endpoints..... 6
 - 4.5 Solution Endpoints.....6
 - 4.6 Assessment Endpoints.....6
 - 4.7 User Management Endpoints..... 7
 - 4.8 System Endpoints..... 7

4.9 Pagination Convention.....	7
5. Authentication & Security.....	7
5.1 Authentication Flow.....	7
5.2 Password Security.....	8
5.3 Role-Based Access Control.....	8
5.4 Rate Limiting.....	8
5.5 Security Headers.....	8
5.6 Input Validation.....	9
6. Frontend Architecture.....	9
6.1 Application Structure.....	9
6.2 State Management.....	9
6.3 User Interface Layout.....	10
6.4 View Components.....	10
6.5 Dashboard Analytics.....	10
7. Configuration & Deployment.....	10
7.1 Environment Variables.....	10
7.2 Vercel Deployment Configuration.....	11
7.3 Health Monitoring.....	11
8. Data Flow & Request Lifecycle.....	11
8.1 Typical Request Flow.....	11
8.2 Database Connection Management.....	12
9. Appendix.....	12
9.1 URL Reference.....	12
9.2 Default Credentials.....	12
9.3 Glossary.....	13

Right-click the table of contents and select “Update Field” to refresh page numbers after opening this document.

1. Introduction

1.1 Purpose of This Document

This document provides a comprehensive technical explanation of the NISAAP (Network Infrastructure Security Assessment & Action Platform) web application. It describes the system architecture, technology stack, database design, API endpoints, authentication mechanisms, security features, and deployment infrastructure. The intended audience includes system administrators, security analysts, developers, and stakeholders at the National Railways of Zimbabwe (NRZ) who need to understand how the platform operates, how data flows through the system, and how various components interact to deliver a secure IoT security assessment experience.

1.2 Platform Overview

NISAAP is a full-stack web application designed to help the National Railways of Zimbabwe manage and monitor the security posture of its IoT (Internet of Things) device infrastructure. The platform enables security teams to register IoT devices, track vulnerabilities, manage remediation solutions, conduct security assessments, and maintain comprehensive audit logs. It provides a role-based access control system that separates administrative functions from analyst and viewer capabilities, ensuring that sensitive operations are restricted to authorized personnel only.

The platform is built as a single-page application (SPA) that communicates with a RESTful API backend. It features a responsive dashboard with real-time statistical charts, data filtering and search capabilities, CSV export functionality, and dark mode support. The entire system is deployed on Vercel's serverless infrastructure with a Turso cloud database, ensuring high availability and global accessibility without the need for on-premise server management.

1.3 Key Features Summary

Feature	Description
IoT Device Management	Register, track, and manage all IoT devices across NRZ network infrastructure including SCADA, PLC, RTU, sensors, cameras, and gateways

Feature	Description
Vulnerability Tracking	Record, categorize, and monitor security vulnerabilities with CVSS scoring, CVE references, and severity classification
Security Solutions	Create and track remediation plans with priority levels, implementation status, cost estimates, and due dates
Security Assessments	Conduct and document formal security assessments with findings, recommendations, and risk ratings
Dashboard Analytics	Real-time visualization of risk distribution, vulnerability severity, device types, and recent activity
Role-Based Access	Four-tier role system (admin, analyst, technician, viewer) with granular permission controls
Audit Logging	Comprehensive tracking of all user actions for accountability and compliance
Rate Limiting	Database-backed brute-force protection on login with automatic IP blocking after 5 failed attempts
Pagination	All list endpoints support page and limit parameters for efficient data retrieval
Dark Mode	Full dark theme support with persistent user preference via local storage

2. System Architecture

2.1 High-Level Architecture

The NISAAP platform follows a modern client-server architecture built on the Next.js full-stack framework. The application uses a single codebase that contains both the frontend user interface and the backend API routes, which are deployed together as a unified application on Vercel's serverless platform. This architecture eliminates the need for separate frontend and backend deployments while maintaining a clear separation of concerns through Next.js's App Router convention.

The frontend is a React-based single-page application rendered in the browser. It communicates with the backend exclusively through RESTful API calls. The backend consists of Next.js API route handlers that process requests, interact with the database

through Prisma ORM, enforce authentication and authorization, and return JSON responses. The database layer uses Turso (a distributed libSQL/SQLite-compatible database) accessed through Prisma's libSQL adapter, providing a serverless-compatible persistence layer that survives Vercel's cold starts.

2.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend Framework	Next.js (App Router)	16.x	Full-stack React framework with server-side rendering and API routes
UI Library	React	19.x	Component-based user interface construction
Styling	Tailwind CSS	4.x	Utility-first CSS framework for responsive design
UI Components	shadcn/ui + Radix	Latest	Pre-built accessible UI components built on Radix primitives
State Management	Zustand	5.x	Lightweight client-side state management
Data Fetching	Native Fetch API	Built-in	HTTP client for API communication
Charts	Recharts	2.x	React charting library for dashboard visualizations
Backend Framework	Next.js API Routes	16.x	Serverless API endpoint handlers
ORM	Prisma	6.x	Type-safe database access with schema migration
Database Driver	@prisma/adapters-libsql	6.x	Prisma adapter for Turso/libSQL connections
Database	Turso (libSQL)	Managed	Distributed SQLite-compatible cloud database

Layer	Technology	Version	Purpose
Validation	Zod	4.x	Schema-based input validation on API routes
Authentication	Custom (HMAC-signed cookies)	N/A	Session-based auth with timing-safe comparison
Rate Limiting	Turso-backed	N/A	Database-persisted rate limiting for serverless
Deployment	Vercel	Managed	Serverless deployment with global CDN
Language	TypeScript	5.x	Type-safe development across the full stack

2.3 Deployment Architecture

The application is deployed on Vercel's serverless platform, which provides automatic scaling, global CDN distribution, and zero-downtime deployments. When a user accesses the website, Vercel serves the pre-rendered HTML shell and JavaScript bundle from the nearest edge location. API requests are routed to serverless function instances that spin up on demand, process the request, connect to the Turso database, and return the response.

The GitHub repository is connected to Vercel, enabling continuous deployment. Every push to the main branch triggers an automatic build and deployment cycle. Environment variables (including database credentials and the session secret) are configured in the Vercel dashboard and injected at build time. The deployment is configured for the Singapore (sin1) region to minimize latency for users in Zimbabwe and Southern Africa, with API routes configured for a maximum execution duration of 30 seconds.

3. Database Design

3.1 Database Technology

The platform uses Turso as its primary database, which is a managed, distributed version of libSQL (an open-source fork of SQLite). Turso was chosen because it provides the simplicity and reliability of SQLite while adding cloud-native features like automatic replication, point-in-time recovery, and global distribution. Unlike traditional SQLite which stores data in a

local file, Turso connects over the network using the `libsql://` protocol, making it compatible with serverless environments like Vercel where persistent local storage is not available.

The Prisma ORM serves as the database access layer, providing type-safe queries, schema migrations, and automatic SQL generation. The `@prisma/adapters-libsql` package bridges Prisma and Turso, translating Prisma queries into libSQL-compatible SQL. The Prisma schema defines the data model, relationships, and indexes, and the Prisma Client is generated from this schema for use throughout the application code.

3.2 Data Model

The database consists of six primary models that represent the core domain entities of the platform. Each model has automatic timestamps (`createdAt`, `updatedAt`), unique identifiers (CUID format), and appropriate indexes for query optimization. The models form a relational hierarchy where IoT devices are the central entity, with vulnerabilities, solutions, and assessments linked to them.

3.2.1 User Model

The User model stores account information for all platform users. Each user has a unique username and email, a hashed password (using `scrypt` with random salt), a role that determines their access level, and optional fields like department and last login timestamp. The role field accepts four values: `admin` (full access including user management and audit logs), `analyst` (read access plus write access to devices, vulnerabilities, solutions, and assessments), `technician` (read access plus write access to devices), and `viewer` (read-only access to all data).

Field	Type	Constraints	Description
<code>id</code>	String (CUID)	Primary Key	Unique identifier
<code>username</code>	String	Unique	Login username
<code>password</code>	String	Required	scrypt-hashed password (salt:key format)
<code>email</code>	String	Unique	User email address
<code>fullName</code>	String	Required	Display name
<code>role</code>	String	Default: <code>analyst</code>	One of: <code>admin</code> , <code>analyst</code> , <code>technician</code> , <code>viewer</code>

Field	Type	Constraints	Description
department	String	Default: IT Security	Organizational department
isActive	Boolean	Default: true	Account status flag
lastLogin	DateTime	Optional	Timestamp of most recent login
createdAt	DateTime	Auto	Record creation timestamp
updatedAt	DateTime	Auto	Last update timestamp

3.2.2 IoTDevice Model

The IoTDevice model represents physical IoT devices deployed across the NRZ network infrastructure. Each device has a unique name, type classification (such as SCADA, PLC, RTU, sensor, camera, gateway, server, workstation, router, switch, or firewall), network information (IP address, MAC address, network segment), physical location data, operational status, and a computed risk level. The risk level is automatically updated when vulnerabilities are created or resolved, ensuring that the device's risk rating always reflects its current security posture.

Field	Type	Constraints	Description
id	String (CUID)	Primary Key	Unique identifier
deviceName	String	Required	Human-readable device name
deviceType	String	Required	Device classification (scada, plc, rtu, sensor, etc.)
ipAddress	String	Required	IPv4 network address
macAddress	String	Optional	MAC address in XX:XX:XX:XX:XX:XX format
location	String	Required	Physical location description
station	String	Optional	Station or substation name

Field	Type	Constraints	Description
status	String	Default: active	One of: active, inactive, maintenance, decommissioned
firmwareVersion	String	Optional	Current firmware version
riskLevel	String	Default: low	One of: critical, high, medium, low
networkSegment	String	Optional	Network segment identifier

3.2.3 Vulnerability Model

The Vulnerability model records security vulnerabilities discovered in IoT devices. Each vulnerability is linked to a specific device, includes a descriptive title and detailed description, severity classification, optional CVSS score (0-10 scale), optional CVE identifier, remediation guidance, and a status tracking field that follows the vulnerability lifecycle from discovery through resolution. Vulnerabilities cascade-delete with their parent device, ensuring data consistency when devices are removed from the system.

Field	Type	Constraints	Description
id	String (CUID)	Primary Key	Unique identifier
deviceId	String	Foreign Key	Reference to the affected IoTDevice
title	String	Required	Vulnerability title
description	String	Required	Detailed vulnerability description
severity	String	Required	One of: critical, high, medium, low
cvssScore	Float	Optional	CVSS base score (0.0 - 10.0)
cveId	String	Optional	CVE identifier (e.g., CVE-2024-1234)
status	String	Default: open	One of: open, in_progress, resolved, accepted_risk

Field	Type	Constraints	Description
discoveredDate	DateTime	Auto	Date vulnerability was discovered
resolvedDate	DateTime	Optional	Date vulnerability was resolved
remediation	String	Optional	Remediation guidance

3.2.4 SecuritySolution Model

The SecuritySolution model tracks remediation actions and security improvements for identified vulnerabilities. Each solution is linked to a specific vulnerability and includes an implementation status lifecycle (proposed, approved, in_progress, implemented, verified), priority classification, optional assignment and due date tracking, cost estimation, and automatic completion date recording when the status reaches implemented or verified.

Field	Type	Constraints	Description
id	String (CUID)	Primary Key	Unique identifier
vulnerabilityId	String	Foreign Key	Reference to the parent Vulnerability
title	String	Required	Solution title
description	String	Required	Detailed solution description
implementationStatus	String	Default: proposed	proposed, approved, in_progress, implemented, verified
priority	String	Default: medium	One of: critical, high, medium, low
assignedTo	String	Optional	Person or team assigned
dueDate	DateTime	Optional	Target completion date
completedDate	DateTime	Auto	Automatically set when status = implemented/verified
costEstimate	String	Optional	Estimated cost of implementation

3.2.5 Assessment Model

The Assessment model documents formal security assessments conducted on IoT devices. Each assessment records the assessor (linked to the User model), the device being assessed, detailed findings and recommendations, and an overall risk rating. Assessments provide a snapshot of a device's security posture at a specific point in time and serve as the official record for compliance and audit purposes.

Field	Type	Constraints	Description
id	String (CUID)	Primary Key	Unique identifier
deviceId	String	Foreign Key	Reference to the assessed IoTDevice
assessorId	String	Foreign Key	Reference to the User who conducted the assessment
findings	String	Required	Detailed assessment findings
recommendations	String	Required	Recommended actions based on findings
riskRating	String	Default: medium	One of: critical, high, medium, low
assessmentDate	DateTime	Auto	Date assessment was conducted

3.2.6 AuditLog Model

The AuditLog model provides a comprehensive record of all user actions within the platform, supporting accountability, compliance, and forensic analysis. Every create, update, delete, login, and login failure event generates an audit log entry. The log captures who performed the action, what action was taken, which module was affected, and optional details. Audit logs are read-only and cannot be modified or deleted through the application interface.

Field	Type	Constraints	Description
id	String (CUID)	Primary Key	Unique identifier
userId	String	Foreign Key (optional)	Reference to the User who performed the action

Field	Type	Constraints	Description
action	String	Required	Action type (e.g., LOGIN, CREATE_DEVICE, UPDATE_VULNERABILITY)
module	String	Required	Module affected (e.g., Authentication, Devices, Vulnerabilities)
details	String	Optional	Human-readable description of the action
ipAddress	String	Optional	IP address from which the action was performed
timestamp	DateTime	Auto	Exact time the action occurred

3.3 Entity Relationships

The data model follows a clear relational hierarchy. A User can create multiple Assessments (one-to-many) and generates multiple AuditLog entries. An IoTDevice can have multiple Vulnerabilities, multiple Assessments, and its risk level is automatically computed from its unresolved vulnerabilities. Each Vulnerability can have multiple SecuritySolutions tracking different remediation approaches. Cascade deletes propagate from IoTDevice to Vulnerability and from Vulnerability to SecuritySolution, ensuring referential integrity when parent records are removed.

4. API Endpoints

4.1 API Architecture

The NISAAP API follows RESTful conventions using Next.js API routes. All endpoints are located under the /api path prefix and return JSON responses with a consistent structure. Successful responses include a success: true field along with the data payload, while error responses include success: false with an error message. All list endpoints support pagination via page and limit query parameters, and most support filtering and search. Authentication is enforced via session cookies on all endpoints except login and health check.

4.2 Authentication Endpoints

Endpoint	Method	Auth Required	Description
/api/auth/login	POST	No	Authenticate user with username/password, returns user data and sets session cookie
/api/auth/logout	POST	No	Clear session cookie, logs the user out
/api/auth/session	GET	No	Check current session validity, returns user data if authenticated
/api/auth/change-password	POST	Yes	Change the current user's password (requires current password verification)

4.3 Device Endpoints

Endpoint	Method	Auth Required	Min Role	Description
/api/devices	GET	Yes	Viewer	List all IoT devices with pagination, filtering by status/riskLevel/deviceType, and search
/api/devices	POST	Yes	Analyst	Create a new IoT device with validated fields
/api/devices/[id]	GET	Yes	Viewer	Get a single device by ID with vulnerability count
/api/devices/[id]	PUT	Yes	Analyst	Update device fields with Zod validation

Endpoint	Method	Auth Required	Min Role	Description
/api/devices/[id]	DELETE	Yes	Admin	Delete a device and all its cascading records

4.4 Vulnerability Endpoints

Endpoint	Method	Auth Required	Min Role	Description
/api/vulnerabilities	GET	Yes	Viewer	List vulnerabilities with pagination, filtering by severity/status/deviceId, and search
/api/vulnerabilities	POST	Yes	Analyst	Create a vulnerability; auto-updates device risk level
/api/vulnerabilities/[id]	GET	Yes	Viewer	Get a single vulnerability with device details
/api/vulnerabilities/[id]	PUT	Yes	Analyst	Update vulnerability fields; re-evaluates device risk
/api/vulnerabilities/[id]	DELETE	Yes	Admin	Delete a vulnerability and its solutions

4.5 Solution Endpoints

Endpoint	Method	Auth Required	Min Role	Description
/api/solutions	GET	Yes	Viewer	List solutions with pagination, filtering by status/priority/vulnerabilityId, and search

Endpoint	Method	Auth Required	Min Role	Description
/api/solutions	POST	Yes	Analyst	Create a new security solution for a vulnerability
/api/solutions/[id]	GET	Yes	Viewer	Get a single solution with vulnerability details
/api/solutions/[id]	PUT	Yes	Analyst	Update solution; auto-sets completedDate when status reaches implemented/verified
/api/solutions/[id]	DELETE	Yes	Admin	Delete a solution

4.6 Assessment Endpoints

Endpoint	Method	Auth Required	Min Role	Description
/api/assessments	GET	Yes	Viewer	List assessments with pagination, filtering by riskRating/device Id, and search
/api/assessments	POST	Yes	Analyst	Create a new security assessment
/api/assessments/[id]	GET	Yes	Viewer	Get a single assessment with device and assessor details
/api/assessments/[id]	PUT	Yes	Analyst	Update assessment findings, recommendations , or risk rating
/api/assessments/[id]	DELETE	Yes	Admin	Delete an assessment

4.7 User Management Endpoints

Endpoint	Method	Auth Required	Min Role	Description
/api/users	GET	Yes	Admin	List users with pagination, filtering by role/status, and search
/api/users	POST	Yes	Admin	Create a new user account with validated fields and password strength check
/api/users/[id]	GET	Yes	Admin	Get a single user's details (password excluded)
/api/users/[id]	PUT	Yes	Admin	Update user profile, role, or status
/api/users/[id]	DELETE	Yes	Admin	Delete a user account

4.8 System Endpoints

Endpoint	Method	Auth Required	Description
/api/dashboard/stats	GET	Yes	Dashboard statistics: device counts, vulnerability distribution, risk overview, recent activity
/api/audit-logs	GET	Yes (Admin)	Audit log entries with pagination, filtering by module/action/userId
/api/health	GET	No	System health check: database connectivity, environment variable status

4.9 Pagination Convention

All list endpoints follow a consistent pagination convention. Clients pass page (default: 1) and limit (default: 50, max: 100 for most endpoints, 500 for audit logs) as query parameters. The response includes a pagination object with the current page, limit, total record count, and total number of pages. This enables efficient data retrieval and prevents memory issues when the database grows large. The `safeParseInt` utility ensures that invalid or malicious query parameter values are clamped to safe defaults.

5. Authentication & Security

5.1 Authentication Flow

NISAAP uses a custom session-based authentication system built on cryptographically signed cookies. When a user submits valid credentials to the `/api/auth/login` endpoint, the server verifies the password hash using Node.js's `scrypt` function with timing-safe comparison, creates a session payload containing the user ID, username, and role, signs the payload using HMAC-SHA256 with the `SESSION_SECRET` environment variable, and sets the signed value as an `HttpOnly` cookie on the response. The cookie is configured with `Secure` flag in production, `SameSite=Lax` to prevent CSRF, and a 24-hour maximum age.

On subsequent requests, the middleware extracts the session cookie, splits the payload from the signature, recomputes the expected signature using the same HMAC key, and verifies it using timing-safe equality comparison (`crypto.timingSafeEqual`). This prevents timing side-channel attacks that could reveal information about the signature. If the signature is valid, the session data is made available to the API route handler for authorization checks.

5.2 Password Security

Passwords are hashed using Node.js's built-in `scrypt` key derivation function with a 16-byte random salt and 64-byte key length. The `scrypt` algorithm is specifically designed to be computationally expensive and memory-hard, making it resistant to brute-force attacks even with modern GPU hardware. The salt and derived key are stored in the format `salt:key` in the database. Password verification uses `timingSafeEqual` to compare the stored key with a freshly derived key, preventing timing attacks that could reveal partial password matches.

5.3 Role-Based Access Control

The platform implements a four-tier role hierarchy that governs what actions each user can perform. The `checkAuth` middleware function enforces these restrictions on every protected API endpoint. The roles and their capabilities are summarized below:

Role	Devices	Vulnerabilities	Solutions	Assessments	Users	Audit Logs
Admin	Read + Write	Read + Write	Read + Write	Read + Write	Full CRUD	Read
Analyst	Read + Write	Read + Write	Read + Write	Read + Write	No Access	No Access
Technician	Read + Write	Read Only	Read Only	Read Only	No Access	No Access
Viewer	Read Only	Read Only	Read Only	Read Only	No Access	No Access

Additionally, only Admin users can delete records and access the Users and Audit Logs sections in the user interface. The frontend enforces these restrictions by hiding navigation items and redirecting unauthorized users, while the backend independently verifies permissions on every API request.

5.4 Rate Limiting

The platform implements brute-force protection on the login endpoint using a database-backed rate limiter. Unlike traditional in-memory rate limiters that lose state on serverless cold starts, the NISAAP rate limiter persists failed login attempt counts in the `_rate_limits` table within the Turso database. After 5 failed login attempts from a single IP address, that IP is blocked for 15 minutes. Successful logins reset the counter, and expired blocks are automatically cleaned up. The rate limiter uses the `x-forwarded-for` header to identify client IPs behind Vercel's CDN.

5.5 Security Headers

The application enforces a comprehensive set of HTTP security headers through Next.js middleware and Vercel configuration. These headers protect against common web vulnerabilities including cross-site scripting (XSS), clickjacking, MIME type sniffing, and unauthorized access to device cameras and microphones. The `Content-Security-Policy` header restricts resource loading to trusted sources while allowing the specific inline styles and scripts required by the React application.

Header	Value	Protection Against
X-Content-Type-Options	nosniff	MIME type sniffing attacks
X-Frame-Options	DENY	Clickjacking (iframe embedding)
X-XSS-Protection	1; mode=block	Reflected XSS attacks in older browsers
Referrer-Policy	strict-origin-when-cross-origin	Information leakage via referrer headers
Permissions-Policy	camera=(), microphone=(), geolocation=()	Unauthorized access to device hardware
Content-Security-Policy	Custom policy	XSS, data injection, unauthorized resource loading
Strict-Transport-Security	max-age=63072000 (Vercel default)	Protocol downgrade and SSL stripping

5.6 Input Validation

All API endpoints that accept user input enforce validation using Zod schemas. The validation schemas are defined in the validations module and cover all create and update operations. The login schema requires non-empty username and password fields. The create user schema enforces minimum username length (3 characters), password strength requirements (minimum 8 characters with at least one uppercase letter, one lowercase letter, and one digit), valid email format, and role enumeration. Device schemas validate IPv4 address format, MAC address format (optional), and device type enumeration. Vulnerability, solution, and assessment schemas enforce minimum content lengths and enumeration constraints for classification fields.

6. Frontend Architecture

6.1 Application Structure

The frontend is a single-page application (SPA) built with React 19 and managed through the Next.js App Router. The entire application renders from a single page component (src/app/page.tsx) that handles session checking, authentication state, and view routing. When a user first loads the page, the application checks for an existing session cookie by calling /api/auth/session. If a valid session exists, the user is taken directly to the dashboard; otherwise, the login view is displayed.

6.2 State Management

Client-side state is managed using Zustand, a lightweight state management library. The application store (useAppStore) maintains the following global state: currentUser (the logged-in user object or null), currentView (which view is currently displayed), sidebarOpen (sidebar toggle state), and darkMode (theme preference). The store persists dark mode preference to localStorage, ensuring the theme selection survives page reloads and browser restarts. View transitions are handled by updating the currentView state, which triggers a re-render of the main AppView component to display the appropriate view.

6.3 User Interface Layout

The application layout consists of a fixed sidebar on the left side and a main content area on the right. The sidebar displays the NISAAP branding, navigation links organized into Main and Administration sections, user information, and a sign-out button. On mobile devices, the sidebar is hidden by default and can be toggled via a hamburger menu button in the top bar. The main content area includes a top bar with the current view title, dark mode toggle, and user profile button, followed by the active view component.

6.4 View Components

The application is composed of specialized view components, each responsible for a specific functional area. The DashboardView displays statistical cards, pie charts, bar charts, and a recent activity feed. The DevicesView, VulnerabilitiesView, SolutionsView, and AssessmentsView provide full CRUD operations with data tables, search, filtering, and modal forms. The UsersView (admin-only) manages user accounts with role assignment. The AuditLogsView (admin-only) displays a searchable log of all user actions. The ProfileView allows users to view and update their profile and change their password.

6.5 Dashboard Analytics

The dashboard is the default landing page after login and provides a high-level overview of the security posture. It uses the Recharts library to render three interactive charts: a vulnerability severity distribution pie chart, a device risk overview bar chart, and a device types distribution donut chart. Below the charts, horizontal progress bars visualize the relative distribution of device risk levels, and a scrollable recent activity feed shows the last 10 audit log entries. All dashboard data is fetched in a single API call to /api/dashboard/stats, which uses Promise.all to execute 19 independent database queries in parallel for optimal performance.

7. Configuration & Deployment

7.1 Environment Variables

The application requires the following environment variables to be configured in the Vercel dashboard or in a local `.env` file for development. These variables are critical for database connectivity, session security, and proper application behavior. Missing variables in the production environment will cause the application to fail on startup with a descriptive error message.

Variable	Required	Description
TURSO_DATABASE_URL	Yes	libsql:// connection URL for the Turso cloud database (e.g., libsql://nisaap-db.xxx.turso.io)
TURSO_AUTH_TOKEN	Yes	Authentication token for the Turso database, obtained from the Turso dashboard
DATABASE_URL	Yes	file: URL for Prisma CLI operations (local development) or placeholder for production
SESSION_SECRET	Yes (Prod)	HMAC signing key for session cookies; must be at least 32 characters; auto-generated in development

7.2 Vercel Deployment Configuration

The `vercel.json` configuration file specifies the build command (`prisma generate` followed by `next build`), the deployment region (`sin1` for Singapore), and a maximum function duration of 30 seconds for API routes. The framework is set to `nextjs` for automatic detection and optimization. Additionally, the configuration includes security headers applied at the Vercel edge level as a backup to the middleware-based headers, providing defense in depth.

The deployment pipeline is triggered automatically when changes are pushed to the main branch of the connected GitHub repository. Vercel builds the application, runs Prisma client generation, and deploys the output to its global CDN. The entire process typically completes in under two minutes. Rollback to a previous deployment is possible through the Vercel dashboard if a new deployment introduces issues.

7.3 Health Monitoring

The `/api/health` endpoint provides a simple health check that verifies database connectivity and environment variable configuration. It returns a JSON response with the current status (ok or degraded), timestamp, environment mode, and boolean flags indicating whether Turso credentials and the session secret are properly configured. This endpoint can be used by monitoring services to detect and alert on service degradation. It does not require authentication, making it accessible from external monitoring tools.

8. Data Flow & Request Lifecycle

8.1 Typical Request Flow

When a user interacts with the application, the following sequence occurs: The user's action (clicking a button, submitting a form, or navigating) triggers a JavaScript event handler in the React component. The handler calls the `apiFetch` utility function, which constructs a fetch request to the appropriate API endpoint with the necessary HTTP method, headers, and body. The request travels over HTTPS to Vercel's CDN, which routes it to the nearest serverless function instance.

The Next.js API route handler receives the request and executes the following pipeline: First, the `checkAuth` middleware extracts and validates the session cookie from the request headers. If the session is invalid or missing, a 401 Unauthorized response is returned immediately. Second, role-based authorization is checked against the `requireAdmin` or `requireWrite` flags on the endpoint. Third, the request body is parsed and validated against the appropriate Zod schema. Fourth, the Prisma Client executes the database query against the Turso database. Fifth, the result is serialized to JSON and returned with the appropriate HTTP status code. The React component receives the response and updates the UI accordingly.

8.2 Database Connection Management

In the Vercel serverless environment, each cold start creates a fresh Prisma Client instance that establishes a new connection to the Turso database. The connection uses the `PrismaLibSQL` adapter with the Turso URL and authentication token. In development mode, the Prisma Client instance is cached on the global object to prevent connection pool exhaustion from hot-reloading. The database connection is non-pooling by design, as Turso manages connection pooling on its side, and each serverless function invocation creates a short-lived connection that is closed when the function completes.

9. Appendix

9.1 URL Reference

Resource	URL
Live Application	https://nisaap-official1.vercel.app
GitHub Repository	https://github.com/Dimplex1423/NISAAP-official
Turso Database Dashboard	https://turso.tech/app
Vercel Deployment Dashboard	https://vercel.com/dashboard
Health Check Endpoint	https://nisaap-official1.vercel.app/api/health

9.2 Default Credentials

The platform is seeded with an initial administrator account for first-time setup. The default credentials are username: munyaradzi and password: dimplex. It is strongly recommended to change the default password immediately after the first login using the Change Password feature accessible from the Profile view. Additional user accounts should be created with appropriately scoped roles following the principle of least privilege.

9.3 Glossary

Term	Definition
NISAAP	Network Infrastructure Security Assessment & Action Platform
NRZ	National Railways of Zimbabwe
IoT	Internet of Things — network-connected physical devices with sensors and software
SCADA	Supervisory Control and Data Acquisition — industrial control system
PLC	Programmable Logic Controller — industrial automation controller
RTU	Remote Terminal Unit — microprocessor-controlled device for remote monitoring
CVSS	Common Vulnerability Scoring System — standardized vulnerability severity measurement

Term	Definition
CVE	Common Vulnerabilities and Exposures — unique identifier for known vulnerabilities
HMAC	Hash-based Message Authentication Code — cryptographic signature algorithm
SPA	Single-Page Application — web app that loads a single HTML page and dynamically updates
CRUD	Create, Read, Update, Delete — the four basic database operations
ORM	Object-Relational Mapping — technique for querying databases using object-oriented code
libSQL	Open-source fork of SQLite with network connectivity and replication support
CUID	Collision-resistant Unique Identifier — compact, URL-safe unique ID format